Statistics 8330 Practice Final Exam

Python/R Computational Statistics Review

Practice Version

In order to receive full credit, your code must include comments that explain what you are doing. You may use R or Python for all problems. Use simulated data unless a problem explicitly says otherwise. There will be partial credit for successfully completing portions of problems, even if you do not complete everything. Each problem is worth equal points.

This is a practice exam designed to resemble prior STAT 8330 final exams. Before using any notes, code, or AI-created materials during an actual exam, confirm the current course rules.

1. **Regression model assessment, smoothing, and cross-validation.**

   In this problem, you will write a function to compare several numeric prediction methods on a one-predictor nonlinear regression problem.

   First, simulate a data set with one predictor and one numeric response. Your simulation should have a smooth nonlinear mean function, such as

   $$Y = 2 + \sin(3X) + \epsilon,$$

   where X is generated from a continuous distribution and ϵ is random noise. Use a fixed random seed.

   Write a function of the form

   ```
   compare_regression_methods(dat, degree_vec, spline_df_vec, span_vec, k=5, makePlot=
       True)
   ```

   where:

   - `dat` is a data frame whose first column is the predictor and whose second column is the response,
   - `degree_vec` is a vector of polynomial degrees to test,
   - `spline_df_vec` or an equivalent argument gives the spline complexities to test,
   - `span_vec` gives LOESS/LOWESS span values to test,
   - `k` is the number of folds for k-fold cross-validation,
   - `makePlot` determines whether the function makes a plot.

   Your function should do the following:

   (a) use k-fold cross-validation to estimate prediction MSE for each candidate method and tuning value;

   (b) compare polynomial regression, spline regression, and LOESS/LOWESS or another local smoother;

   (c) return a table with method name, tuning value, and mean cross-validation MSE;

(d) identify the method and tuning value with the lowest cross-validation MSE;

(e) if `makePlot=TRUE`, plot the simulated data and overlay the fitted curve from the selected method.

Demonstrate your function on simulated data. Briefly explain which method was selected and what the tuning parameter means. Your explanation should mention the bias-variance tradeoff and why cross-validation is being used instead of training MSE.

2. **Lasso penalty simulation study.**

In this problem, you will conduct a simulation study of how lasso prediction error depends on the penalty parameter.

Create at least three simulation scenarios. Your scenarios should be chosen so that different penalty strengths might be favored. For example, you might vary:

- sparse signal versus dense signal,
- low noise versus high noise,
- small sample size versus larger sample size,
- many irrelevant predictors versus few irrelevant predictors.

Write code that does the following:

(a) simulates training and test data for each scenario;

(b) fits lasso regression over a grid of penalty values;

(c) standardizes predictors before fitting the lasso;

(d) computes prediction error on test data for every replicate and penalty value;

(e) repeats the process over many simulation replicates;

(f) produces a plot of mean test MSE versus penalty value for each scenario.

Your output should include:

- a short table giving the best penalty value in each scenario,
- at least one plot showing prediction error as a function of the lasso penalty,
- a brief written explanation of why larger or smaller penalties were favored in your scenarios.

Your explanation should discuss what the lasso penalty does to coefficients, why standardization matters, and how the results illustrate the bias-variance tradeoff.

3. **Classification simulation, metrics, and tuning.**

In this problem, you will compare classification methods using simulated data.

Simulate at least two binary classification scenarios:

- one scenario where a linear boundary should work reasonably well;
- one scenario where the true boundary is nonlinear.

Write a function of the form

```
classification_study(scenario, nrep, train_size, test_size, makePlot=True)
```

or a similar function with clearly documented arguments.

Your function should compare at least four of the following methods:

- logistic regression,
- LDA,
- QDA,
- K-nearest neighbors,
- decision tree with pruning or depth control,
- random forest,
- AdaBoost,
- support vector machine.

Your study should do the following:

(a) fit each classifier on training data only;

(b) compute test accuracy for each replicate;

(c) for at least one method, tune a parameter by cross-validation inside the training data, such as KNN's k, tree pruning parameter, or SVM's C and γ;

(d) report a table of mean test accuracy by method and scenario;

(e) report a confusion matrix for the best method in one replicate;

(f) make a plot such as a boxplot of test accuracies by method.

Briefly explain your results. Your explanation should mention which methods are expected to work better for linear versus nonlinear class boundaries, why stratified splitting or stratified cross-validation is useful, and why training accuracy alone is not sufficient.

4. **Bootstrap confidence intervals and permutation tests.**

In this problem, you will write two simulation-based inference functions.

Part A. Bootstrap confidence interval.

Write a function of the form

```
bootstrap_ci(data, statistic_function, n_boot=2000, confidence=0.95)
```

that returns a percentile bootstrap confidence interval for a statistic. Demonstrate the function on at least one of the following:

- a confidence interval for a mean,
- a confidence interval for a median,
- a confidence interval for a regression slope by resampling rows.

Your output should include the point estimate, lower endpoint, upper endpoint, and a plot of the bootstrap distribution.

Part B. Permutation test.

Write a function of the form

```
permutation_test_two_groups(group1, group2, statistic_function, n_perm=5000,
    alternative="two-sided")
```

that performs a permutation test comparing two groups. Demonstrate your function on simulated two-group data.

Your permutation-test output should include:

- the observed statistic,
- the simulated p-value,
- a plot of the permutation null distribution with the observed statistic marked.

Briefly explain the difference between bootstrap resampling and permutation testing. Your explanation should clearly state what exchangeability means and when a permutation test is appropriate.

5. **Unsupervised learning: PCA, k-means, and hierarchical clustering.**

   In this problem, you will perform an unsupervised learning analysis using simulated multivariate data.

   Simulate a data set with several numeric variables and an underlying cluster structure. You may keep the true cluster labels for checking recovery, but your unsupervised methods should not use the labels when fitting.

   Your analysis should include the following:

   (a) standardize the predictor variables;

   (b) run PCA and report the proportion of variance explained by the first few principal components;

   (c) make a scree plot;

   (d) make a two-dimensional PCA score plot;

   (e) run k-means clustering for several values of k;

   (f) make an elbow plot or use another reasonable diagnostic for choosing k;

   (g) run hierarchical clustering and display a dendrogram;

   (h) compare the k-means and hierarchical clustering results qualitatively.

   If you simulated true cluster labels, compute a recovery measure such as an adjusted Rand index or a cross-tabulation of true labels and recovered clusters.

   Briefly explain:

   - why variables should usually be standardized before PCA or clustering;
   - the difference between PCA and clustering;
   - why unsupervised learning results should be interpreted cautiously;
   - how cluster overlap or non-spherical clusters could cause k-means to fail.